

ВСП тема 2 задание 2.2 часть 2

Справочник по математическим объектам и их представлению в Scilab

В среде Scilab встроены типы данных, которые в большинстве случаев являются основой для проведения математических расчетов. Мы будем называть их **объектами**. Внутренняя структура объектов заранее предопределена внутри среды и скрыта от глаз пользователей. Работу с памятью при обработке объектов среда также берет на себя. В дальнейшем эти предопределенные объекты могут послужить для создания пользовательских объектов.

В основе своей объекты представляют собой массивы, в которых сохранены определенные данные. В этом разделе мы научимся создавать и манипулировать основными предопределенными объектами, без которых не обходится ни один из сеансов работы с программой.

Переменная в Scilab (англ. *Variable*) — это именованный массив всего с одним полем, которое хранит данные некоторого типа.

Создать переменную в среде не составляет труда. Для этого, как вы уже знаете, достаточно ввести ее имя и присвоить ей какое-либо начальное значение. Для переменной будет автоматически выделено место в памяти, а ее область видимости по умолчанию становится локальной.

Некоторые правила, которым должны удовлетворять имена переменных и вообще любых объектов среды:

- Имя переменной может состоять из букв латинского алфавита (верхнего и нижнего регистра) и цифр;
- Имя переменной не может начинаться с цифры, но может начинаться с символов '%', '_', '#', '!', '\$', '?';
- Регистр в имени играет роль, то есть переменные с именами var, VAR, Var и т. п. разные;
- Запрещено совпадение имени переменной с зарезервированными словами, такими как имена объявленных функций, констант и др.;

Для начала попробуем создать несколько переменных

```
--> n1=25, n2=65.33, n3=-36.4e-6, n4=9+%i*4
```

В результате мы получили 4 переменных, 3 из которых хранят вещественные числа и одна — комплексное число. Обратите внимание, как вводится мнимая единица. В среде

Scilab мнимая единица является предопределенной константой с именем 'i'. Знак процента указывает на то, что вы обращаетесь к константе.

Целые числа.

В Scilab целые числа возможно создавать только через специальные функции. Во всех остальных случаях числовому значению всегда будет присваиваться вещественный тип данных.

Для хранения целого числа в памяти может быть использовано разное число битов, а именно 8, 16 и 32. От количества используемых битов зависит диапазон целых чисел. Кроме того, имеется возможность включения и отключения знакового бита, что бывает полезно, когда отрицательные целые числа не требуются.

В таблице ниже приведены функции для создания целых чисел и диапазоны возможных значений. Во всех случаях функция возвращает целое число, указанное в качестве аргумента, с определенным способом его хранения в памяти.

Имя функции	Тип целого числа	Диапазон	Код представления
<code>int8()</code>	знаковое 8-битное число	$[-2^7, 2^7 - 1] = [-128, 127]$	1
<code>uint8()</code>	беззнаковое 8-битное число	$[0, 2^8 - 1] = [0, 255]$	11
<code>int16()</code>	знаковое 16-битное число	$[-2^{15}, 2^{15} - 1] = [-32768, 32767]$	2
<code>uint16()</code>	беззнаковое 16-битное число	$[0, 2^{16} - 1] = [0, 65535]$	12
<code>int32()</code>	знаковое 32-битное число	$[-2^{31}, 2^{31} - 1] = [-2147483648, 2147483647]$	4
<code>uint32()</code>	беззнаковое 32-битное число	$[0, 2^{32} - 1] = [0, 4294967295]$	14

Какой тип целочисленной переменной следует применить, пользователь решает сам. Целые числа могут быть использованы как в чисто математических задачах, так и программировании, в качестве счетчиков.

Для работы с целочисленными переменными существует две функции:

- `iconvert(X, itype)` — меняет представление в памяти в общем случае матрицы из вещественных или целых чисел. Функции при этом следует передавать имя этой матрицы X и код внутреннего представления целого числа itype из последней колонки таблицы выше;
- `inttype(X)` — возвращает код внутреннего представления в общем случае матрицы целочисленных данных.

Матрицы и векторы

Вектор в Scilab — это упорядоченная совокупность элементов (одномерный массив) одного типа данных. Упорядоченность для пользователя в этом смысле проявляется в том, что к каждому элементу вектора можно обратиться по его уникальному порядковому номеру или *индексу*. В среде Scilab все индексы начинаются с

единицы, что немного не привычно, так как например в программировании на языке Си те же индексы массивов начинаются с нуля.

Ранее мы уже не раз создавали вектора и вы уже знаете, что для этого нужно элементы заключить в квадратные скобки, то есть

```
Vector = [e1, e2, e3]; // Вектор с тремя элементами в 6-ти с именами e1, e2 и e3
```

При объявлении вектора совершенно не обязательно отделять их друг от друга запятыми — достаточно простого пробела. Например, следующее объявление не приведет к синтаксической ошибке.

Вектор может быть записан столбцом или строкой. В первом случае он называется вектор-столбцом, а во втором — вектор-строкой. От того, как представлен вектор (столбцом или строкой), зависит применимость некоторых операторов. Например, из линейной алгебры вы знаете что скалярное произведение двух векторов возможно только, если первый множитель представлен строкой, а второй — столбцом.

Выше мы создавали векторы-строки. Векторы-столбцы создаются путем отделения каждого элемента точкой с запятой, то есть

```
-->[1; 2; 3]
ans =
    1.
    2.
    3.
```

Матрица в Scilab — это двумерный массив однотипных элементов. Можно понимать матрицу как несколько векторов-строк, записанных столбцом.

Создать матрицу в Scilab можно одним из нескольких способов:

- Матрицу можно создать из составляющих ее элементов;
- Из имеющихся векторов, упорядочив их строками или столбцами;
- Одной из специальных функций.

В общем случае синтаксическая конструкция имеет вид

$[x_{11}, x_{12}, \dots, x_{1n}; x_{21}, x_{22}, \dots, x_{2n}; \dots; x_{m1}, x_{m2}, \dots, x_{mn}]$

Таким образом, вы создаете векторы-строки, которые отделяете точкой с запятой. Запятая в этом случае, как и с вектором, не обязательна.

```
-->A=[1 2; 3 4] // создадим матрицу из составляющих ее элементов
A =
    1.    2.
    3.    4.
```

Функции

В Scilab есть predefined математические функции, такие как тригонометрические, экспонента и другие. Но что, если вам необходимо определить собственную функцию? Далее мы будем отличать математические функции и программируемые функции. Разница между ними заключается в том, что программируемые функции призваны реализовывать некоторый алгоритм, в то время как математическая функция отражает связь между множеством аргументов и множеством значений функции.

Отметим, что математическую функцию можно объявить через программирование, но так как обычно ее тело состоит из одной строки, то рациональнее всего объявлять ее через специальную функцию **deff()**. Общий синтаксис имеет вид

```
deff('[Y1,Y2...]=Fname(X1,X2,...)', ['Y1=выражение_1'; 'Y2=выражение_2;...'])
// [Y1,Y2,...] – вектор возвращаемых переменных (имена назначаете сами)
// Fname – имя функции, которое вы назначаете сами
// (X1,X2,...) – список из аргументов функции
// 'Y1=выражение_1;Y2=выражение_2;...' – для каждой выходной переменной должно быть определено выражение,
// которое может зависеть от аргументов, а может и не зависеть.
```

Пример использования

```
--> deff('y=f(x)', 'y=x*x-2')
--> x=3; y=f(x)
y =
    7.
```

Также следует обратить внимание на функцию **funcprot()** (от англ. *function protection*). С помощью **funcprot()** вы можете защитить ваши функции от случайного их переопределения. Функция принимает один целочисленный аргумент:

- 0 — отключение механизма защиты;
- 1 — формирование предупреждения (используется по умолчанию);
- 2 — формирует ошибку при попытке переопределения существующей функции.

Вообще говоря, при определении функции через **deff()** она может иметь сколь угодно много инструкций, которые необходимо записывать в вектор. Однако, запись становится неудобочитаемой и в этом случае целесообразнее использовать уже вторую конструкцию **function...endfunction**. Общий синтаксис выглядит так

```
function <выходные переменные>=имя_функции(аргументы)
...
инструкции
...
endfunction
```

Данная конструкция, как правило, используется для написания пользовательских программируемых функций, однако, в зависимости от предпочтений пользователя, она может использоваться и вместо функции **deff()**. Использовать эту конструкцию можно в интерпретаторе, который воспринимает ее по особому. Сначала необходимо ввести первую строку с ключевым словом *function*, именем функции, аргументами и выходными переменными. Далее вы можете нажать <Enter> и интерпретатор будет воспринимать все, что вы вводите, как инструкции для данной функции до тех пор, пока вы не введете ключевое слово *endfunction*. Если вы не допустили синтаксических ошибок в теле функции, то она будет готова к использованию, в противном случае интерпретатор выведет вам ошибку и объявлять придется заново.